Week 4

Search II: Local Search, Restarts and Learning

◎ Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** explain local search as an alternative to systematic search for CSPs: state the min-conflicts heuristic, describe how tabu search and simulated annealing help a local search escape a local minimum, and explain why local search sacrifices completeness (it cannot prove a CSP infeasible) for speed on large instances;
- **(H2)** explain and apply large neighbourhood search (LNS): describe the destroy/repair loop, select a reasonable neighbourhood (which variables to relax) for a given problem, and implement an LNS loop that uses OR-Tools CP-SAT as the repair engine to improve an existing solution;
- **(H3)** explain how restarts combined with nogood (clause) learning let a search avoid repeating an already-discovered failure, and describe, at a conceptual level, how lazy clause generation combines this idea with constraint propagation to make CP-SAT's search efficient;
- **(H4)** configure and run a portfolio/parallel search in OR-Tools CP-SAT (`num_search_workers`), and explain, in terms of Objectives H1–H3, what kind of search strategy each parallel worker is contributing;
- **(H5)** given a CSP/optimization instance that a naive Week-3-style systematic search struggles to solve in a practical amount of time, select and justify a combination of this week's techniques (local search, LNS, restarts with learning, portfolio search), implement it in CP-SAT, and verify – from real solver output – that it produces a solution, or an improved solution, within a stated time budget.

Week 3 explored the search tree exhaustively, node by node, guided by ordering heuristics and pruned by look-ahead. This chapter asks what happens once that tree becomes too large to explore exhaustively at all. Four units, taken together, answer this chapter's single running question – why is OR-Tools CP-SAT fast on instances a plain Week 3 backtracker chokes on? – one mechanism at a time. Unit 1 abandons the search tree entirely: local search starts from a complete guess and repeatedly repairs it. Unit 2 generalizes that same repair habit from one variable at a time to

a whole neighbourhood at once, using CP-SAT itself as the repair engine. Unit 3 returns to systematic search and asks what a smart search does immediately after a failure: restart, remembering what just went wrong, rather than restart blind. Unit 4 closes the chapter by opening a knob you already used without understanding it – Week 3's own `num_search_workers` – revealing it as several of Units 1–3's strategies running concurrently, sharing what each one learns.

N-queens makes its fourth problem-pool appearance this chapter, scaled up and reframed as a repair problem: a complete assignment with a violated-constraint count to drive toward zero, rather than the pure hard-constraint satisfaction model Weeks 1–3 built. The Golomb ruler joins the pool for the first time, posed as a genuine minimization problem – the shortest ruler for a given number of marks – giving Units 2–4 a second, complementary demonstration where `model.Minimize(...)` targets a real objective rather than a violated-constraint count.

This chapter stays deliberately narrow in a few places. It adds no propagation mechanics beyond what Week 2 already covered; CP-SAT's own propagation is taken as a given here, exactly as Week 3 took it. It re-teaches no systematic-search algorithm detail beyond what is needed as a contrast reference point – backtracking, the search tree, variable and value ordering, forward checking and MAC remain Week 3's content, recalled here by name only. It develops no symmetry breaking and no tree decomposition or treewidth theory; both belong to Week 5, which this chapter's own transfer notes point forward to explicitly. It introduces no MiniZinc and no Gurobi – this chapter, like every chapter before it, stays CP-SAT only. And it develops no formal constraint-optimization theory: weighted CSP, Max-CSP and penalty variables are Week 8's job. `model.Minimize(...)` and an informal "how many constraints does this assignment violate" count are used here only operationally, as the concrete quantity local search (Unit 1) and LNS (Unit 2) have something to improve step by step.

4.1 Local Search for CSPs

1 • Pre-instruction **Why It Matters**

Ask the backtracking search you built in Week 3 to place a million queens on a million-by-million board, and you can wait long past the end of this course for an answer – the search tree alone would not fit in memory. Ask a completely different kind of search – one that starts from a full, bad guess and just keeps nudging it toward fewer conflicts, one queen at a time – and a well-known literature result finds a solution in about a minute. Same problem, a fundamentally different way of searching for an answer.

1 • Pre-instruction **Prior Knowledge**

Week 3's backtracking search builds a search tree node by node: it holds a *partial* assignment, chooses the next unassigned variable, and extends the assignment one variable at a time, guided by variable/value ordering heuristics (MRV, the degree heuristic, dom/wdeg, LCV) and pruned by forward checking or MAC. This unit abandons that tree entirely – there is no partial assignment here, only a complete (possibly invalid) one, repaired step by step; the contrast between the two search shapes is the point of everything that follows.

Every search this course has built so far shares one habit: start from nothing, and add one committed variable at a time, backtracking whenever a partial choice cannot be extended. That habit is exactly what makes Week 3's search tree grow so large on a hard instance – the tree has to represent every partial assignment the search might ever need to consider. This unit throws that habit away completely and starts from the opposite end: begin with every variable already assigned something, even if that something is bad, and repeatedly repair it.

4.1.1 A different search shape

Local search: A search method that starts from a complete (but possibly constraint-violating) assignment and repeatedly modifies it, one step at a time, to reduce the number of violated constraints, instead of building a partial assignment one variable at a time.

Local search is a search method that starts from a *complete* assignment – every variable already has a value, drawn from its domain, but one or more constraints may currently be violated – and repeatedly changes a single variable's value to reduce the number of violations. There is no partial assignment anywhere in this process, no search tree, and, crucially, no way to prove a CSP infeasible: local search can only ever report one of two outcomes – it finds a solution, or it exhausts its step budget – and it can never report that no solution exists at all.

Definition: Local search

A search method that starts from a complete (but possibly constraint-violating) assignment and repeatedly modifies it, one step at a time, to reduce the number of violated constraints, instead of building a partial assignment one variable at a time.

Analogy

Local search resembles rearranging a wedding seating chart one guest at a time. Instead of designing the whole chart from scratch, table by table, you start from a full (possibly bad) seating – every guest already has a seat – and repeatedly move the single most awkwardly placed guest to whichever seat causes the fewest conflicts with the guests already seated nearby, checking again after every move, until, hopefully, no one is unhappy. You never erase the whole chart and start over; you only ever move one guest.

That last point is worth pausing on, since it is exactly what separates this unit from every search Week 3 built: backtracking commits to a value once and only revisits it by explicit backtracking, undoing the most recent choice; local search never "undoes" anything in that sense at all – every variable keeps whatever value it currently holds until the search itself decides to overwrite it, and the search can freely revisit and change a variable it has already touched, as many times as it likes.

4.1.2 The min-conflicts heuristic

min-conflicts heuristic: A local-search repair rule: at each step, pick a variable currently involved in at least one violated constraint, and reassign it to the value, from its own domain, that violates the fewest constraints, breaking ties arbitrarily.

The specific repair rule this unit uses is the **min-conflicts heuristic**: at each step, pick a variable that is currently involved in at least one violated constraint, and reassign it to whichever value in its own domain violates the fewest constraints, breaking ties arbitrarily (with a fixed random seed, so the choice is reproducible in code). Repeat until zero violations remain, or a step budget runs out.

Definition: Min-conflicts heuristic

A local-search repair rule: at each step, pick a variable currently involved in at least one violated constraint, and reassign it to the value, from its own domain, that violates the fewest constraints, breaking ties arbitrarily.

This is the exact algorithm behind a well-known literature result: Minton et al. (1992, cited via Russell and Norvig's *Artificial Intelligence: A Modern Approach*) reported that min-conflicts solves million-queens instances in roughly a minute, on hardware of its time. That claim is not reproduced at that scale by this week's own code – the code below runs on a few thousand queens, a size chosen to keep runtime and memory modest for a course lab, not to reproduce a three-decade-old benchmark on unrelated hardware.

The code below builds a complete random assignment for N-queens (one queen per row, a column chosen at random for each row) and repairs it row by row. Efficient conflict counting matters here: recomputing every queen's conflicts from scratch on every candidate move would cost $O(N)$ per candidate, $O(N^2)$ per step – far too slow for thousands of queens. Instead, three small occupancy counters (one per column, one per each of the two diagonal directions) let every candidate's conflict count be read off in $O(1)$:

```
rng = random.Random(seed)
columns = [rng.randrange(n) for _ in range(n)]
col_count = [0] * n
diag1_count = [0] * (2 * n - 1)
diag2_count = [0] * (2 * n - 1)
for row, col in enumerate(columns):
   col_count[col] += 1
   diag1_count[row - col + n - 1] += 1
   diag2_count[row + col] += 1
```

Given those counters, `min_conflicts_step` evaluates every candidate column for one conflicted row, moves the queen to whichever column has the fewest conflicts (breaking ties randomly), and updates the three counters to match:

```
col_count[columns[row]] -= 1
diag1_count[row - columns[row] + n - 1] -= 1
diag2_count[row + columns[row]] -= 1

best_columns, best_conflicts = [], n + 1
for col in range(n):
   conflicts = col_count[col] + diag1_count[row - col + n - 1] + diag2_count[
       row + col]
   if conflicts < best_conflicts:
      best_columns, best_conflicts = [col], conflicts
   elif conflicts == best_conflicts:
      best_columns.append(col)
new_col = rng.choice(best_columns)

columns[row] = new_col
col_count[new_col] += 1
diag1_count[row - new_col + n - 1] += 1
diag2_count[row + new_col] += 1
```

The outer loop repeatedly picks a currently conflicted row at random and repairs it, stopping the moment zero conflicts remain or the step budget runs out:

```
for step in range(1, max_steps + 1):
```

```
    conflicted = _conflicted_rows(n, columns, col_count, diag1_count,
        diag2_count)
    if not conflicted:
        print(f"Solved N={n}: 0 conflicts remaining after {step - 1} steps.")
        return columns, step - 1
    row = rng.choice(conflicted)
    min_conflicts_step(row, n, columns, col_count, diag1_count, diag2_count,
        rng)
remaining = len(_conflicted_rows(n, columns, col_count, diag1_count,
    diag2_count))
print(f"Stopped N={n}: {remaining} conflicted rows remain after {max_steps}
    steps.")
```

The real captured run, for $N = 2000$ with a fixed seed and a generous 100,000-step budget, reaches zero conflicts well before that budget is exhausted:

Expected Output

```
Solved N=2000: 0 conflicts remaining after 1288 steps.
```

That is not the literature's million-queens claim reproduced – it is a much smaller, honestly captured number on a much smaller instance – but it already makes the same point that claim makes: a handful of thousand repair steps, each touching only one queen, found a zero-conflict assignment for a board size Week 3's own backtracking search would never finish exploring exhaustively.

4.1.3 Getting stuck: the local minimum

local minimum (search): A complete assignment at which no single-variable change strictly reduces the number of violated constraints, even though a true solution – zero violations – may still exist somewhere else in the space of assignments.

Min-conflicts, run by itself, can also get stuck permanently, at a **local minimum (search)**: a complete assignment at which no single-variable reassignment strictly reduces the number of violated constraints, even though a genuine solution – zero violations – may still exist somewhere else in the space of assignments.

Definition: Local minimum (search)

A complete assignment at which no single-variable change strictly reduces the number of violated constraints, even though a true solution (zero violations) may still exist elsewhere in the search space.

Analogy

A local minimum resembles a delivery driver who only ever takes the next street that shortens today's remaining route. Sometimes the only way to a genuinely shorter total route is to first drive one street that *looks* longer – a driver who never accepts a worse-looking turn can get stuck circling the same few streets indefinitely, even though a shorter route exists two turns away.

A common misconception at this point is treating min-conflicts getting stuck as evidence the CSP itself has no solution. It is not: a local minimum is a property of the *current assignment* and the single-variable moves reachable from it, not a property of the whole problem. Because local search never builds a search tree and never proves any branch exhausted, it has no mechanism at all for distinguishing "stuck at a local minimum, a solution exists elsewhere" from "genuinely infeasible." Both look identical from inside a plain min-conflicts run: no violations were ever removed, and no more moves help.

4.1.4 Escaping a local minimum: tabu search and simulated annealing

Two classic extensions give min-conflicts a way out of a local minimum, without abandoning local search's core one-variable-at-a-time habit.

Tabu search: A local-search strategy that temporarily forbids re-selecting a recently changed variable/value combination, kept on a short-term "tabu list," preventing the search from immediately undoing its own last move.

Tabu search keeps a short-term memory – the *tabu list* – that temporarily forbids re-selecting a variable/value combination that was changed in the last few steps. Its specific purpose is stopping the search from immediately undoing its own last move and cycling forever between the same two or three states, a real risk once a search is close to a local minimum and every available move looks equally unattractive.

Analogy

Tabu search resembles a delivery driver's notebook of streets just avoided a jam on, with each one marked off-limits for the next few blocks. It is a short-term memory that stops the exact same reversal from happening again right away, without permanently ruling that street out forever.

Simulated annealing: A local-search strategy that occasionally accepts a strictly worse move, with a probability that starts high and decreases over the run, specifically to escape a local minimum a purely greedy repair rule cannot leave.

Simulated annealing takes a more direct approach: it occasionally accepts a move that makes the violation count strictly *worse*, with a probability that starts high early in the run and decreases as the run continues – an analogy to a physical cooling schedule, hence "temperature." This is a deliberate answer to the local-minimum problem: a purely greedy repair rule, by definition, can never take a worse-looking step, so it can never leave a local minimum on its own; a rule willing to step "uphill" sometimes can walk right out of one.

Analogy

Simulated annealing resembles the same delivery driver, but willing – especially early in the day – to take a longer-looking street on a hunch, less and less often as the day wears on. Early on, the driver experiments freely (a high "temperature"); by evening, only genuine improvements are accepted.

Figure 4.1 draws this contrast schematically – min-conflicts' single greedy move against simulated annealing's willingness to accept one temporarily worse move – as an illustrative diagram, not a result read off any of this week's own code.

Common Pitfall

A tempting but incorrect conclusion after watching min-conflicts solve a hard-looking instance quickly is: "local search is just as good as backtracking, only faster." It is not the same tool with the same guarantees. Backtracking (Week 3) is *complete* – if a CSP has no solution, backtracking eventually proves that by exhausting the root. Local search sacrifices that guarantee entirely: it can run out of steps on a genuinely infeasible instance and on a solvable instance it simply failed to escape a local minimum from, and it has no way to tell you which one just happened. Speed on large instances is exactly what local search trades completeness for – never the other way around.

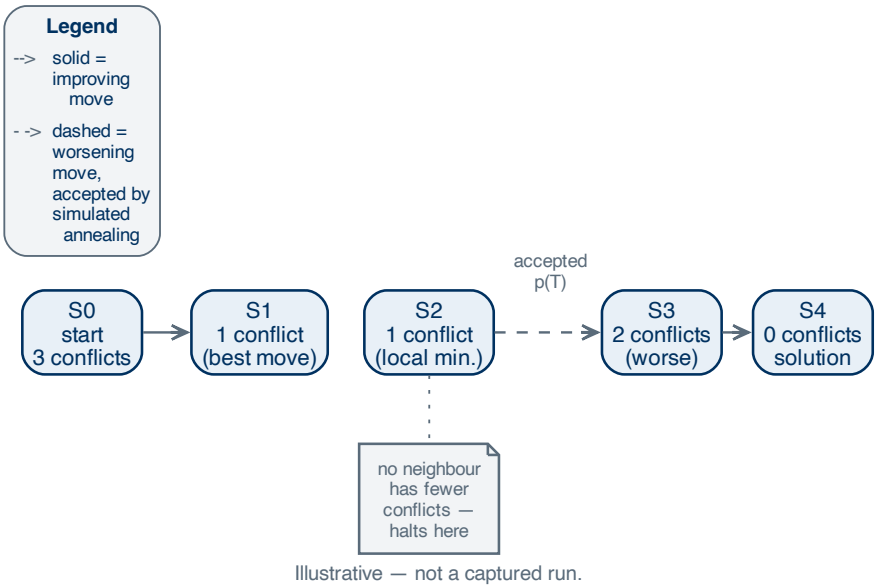


Figure 4.1: A schematic contrast between min-conflicts' single greedy repair move (solid arrows, always improving) and simulated annealing's willingness to accept one temporarily worse move (dashed arrow) to escape a state where every neighbouring move is equal to or worse than the current one. Illustrative, not a captured run.

3 • Participation **Now You Try**

You are running min-conflicts on 4-queens, starting from the worst possible assignment: `Q1=1, Q2=2, Q3=3, Q4=4` (all four queens on the same diagonal – every pair attacks every other pair). `Q2` is selected as the next variable to repair. For each of the 4 possible columns (1–4), count how many of the other three queens – fixed at `Q1=1, Q3=3, Q4=4` – it would conflict with (same column or same diagonal), and state which column(s) min-conflicts would move `Q2` to.

Feedback. Checking `Q2` at each column against the fixed `Q1=1, Q3=3, Q4=4`: column 1 conflicts with `Q1` only (same column) – 1 conflict. Column 2, the current column, conflicts with all three (shares a diagonal with each of `Q1`, `Q3` and `Q4`) – 3 conflicts. Column 3 conflicts with `Q3` only (same column) – 1 conflict. Column 4 conflicts with `Q3` (shared diagonal) and `Q4` (same column) – 2 conflicts. Columns 1 and 3 tie for the minimum, 1 conflict, down from 3 at the current column; min-conflicts moves `Q2` to whichever of the two its tie-break rule picks – either is a strict improvement over staying at column 2.

Check Yourself 4.1

In your own words, explain why min-conflicts is not a systematic search technique, describe one scenario where it can get permanently stuck – a local minimum with no strictly better single-move neighbour – and explain how simulated annealing's acceptance rule is specifically designed to escape such a stuck state.

5 • Follow-through **Transfer — Where Else?**

Min-conflicts and its relatives are not this course's last word on repair-based search. Unit 2 generalizes the same "start from a solution, make one local change, check if it helped" habit to repairing many variables at once, using a real solver behind the repair step instead of a single hand-written rule. Later, Weeks 12 and 14 lean on this same local-search family of ideas – rostering and vehicle routing, respectively – whenever a problem is too large for a Week 3-style systematic search to handle from scratch.

4.2 Large Neighbourhood Search

1 • Pre-instruction **Why It Matters**

Min-conflicts repairs one variable at a time; that is fast, but shortsighted – sometimes the only way out of a bad neighbourhood is to change ten variables together, which a single-variable move can never see happening. Large neighbourhood search keeps local search's core habit – start from a solution, make it better – but hands each repair step to the full CP-SAT solver instead of a single hand-written move.

1 • Pre-instruction **Prior Knowledge**

Unit 1's min-conflicts repair step picks one variable currently involved in a violation and reassigns it to reduce that violation count. This unit generalizes that exact habit – start from a complete assignment, change part of it, check whether it helped – from "repair one variable by hand" to "repair a whole relaxed neighbourhood with a real solver." The destroy/repair vocabulary and the running goal of driving a violation count toward zero both carry over directly.

Unit 1 already sacrificed completeness for speed – min-conflicts never proves anything, it only repairs. This unit keeps that trade, but raises the ceiling on what a single repair step can accomplish, by replacing a one-line hand-written rule with an entire solver call.

4.2.1 The destroy/repair loop

Large neighbourhood search (LNS): An iterative improvement method that, from an existing solution, relaxes (destroys) a chosen subset of variables while fixing the rest, then hands the freed sub-problem to a complete solver to re-optimize (repair) before deciding whether to keep the result.

Large neighbourhood search (LNS) starts from an existing complete assignment – either a genuine solution or a "good enough" assignment with some violations still counted – and picks a subset of variables, the neighbourhood for this round, to work on.

Destroy/repair names the two steps that follow: destroy that neighbourhood by unfixing its variables, leaving every other variable fixed at its current value; then repair it by handing the smaller, freed sub-problem to CP-SAT, which re-optimizes just that neighbourhood while everything else stays put. Keep the result if it is at least as good as before, otherwise revert to the previous assignment. Repeat for a fixed number of rounds, or until a stopping condition – no improvement for several rounds, a time budget, or a proof of optimality on the whole problem.

Which variables end up in that neighbourhood is the single choice this unit spends the most time on below.

Destroy/repair: The two-step core of one LNS iteration – destroy relaxes a chosen neighbourhood

Definition: Large neighbourhood search (LNS)

An iterative improvement method that, from an existing solution, relaxes ("destroys") a chosen subset of variables while fixing the rest, then hands the freed sub-problem to a complete solver to re-optimize ("repair") before deciding whether to keep the result.

Analogy

LNS resembles renovating one room of a house at a time while living in the rest. You do not tear down and rebuild the whole house – that would be Week 3's systematic search, starting over from nothing – and you do not move one piece of furniture at a time either – that would be Unit 1's local search. Instead, you pick a whole room, a neighbourhood of variables, clear it out completely, and let a professional planner – the CP-SAT solver, playing the repair engine's role – work out the best layout for just that room, while every other room in the house stays exactly as it was.

Figure 4.2 lays out this loop as a four-step cycle, worth checking against before either of this unit's two worked scripts.

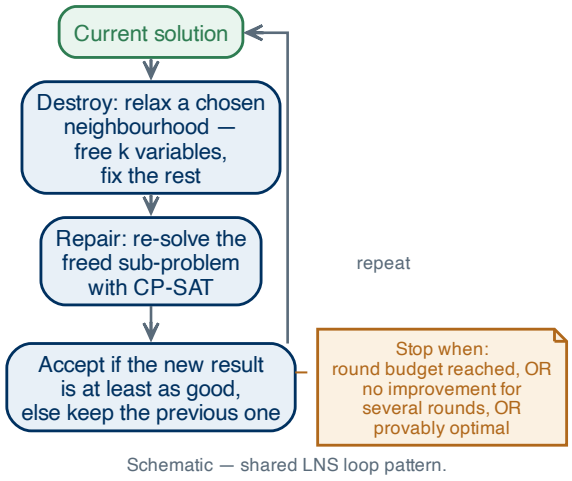


Figure 4.2: The destroy/repair/accept loop shared by this week's large N-queens and Golomb-ruler LNS code, with its stop condition set apart.

4.2.2 LNS as a generalization of min-conflicts

It is worth being explicit about exactly what changed between Unit 1 and this unit, since both share the same destroy/repair skeleton at heart. Min-conflicts destroys and repairs exactly one variable per step, using a one-line hand-written rule – "pick the value with fewest conflicts" – as the repair engine. LNS destroys and repairs a whole chunk of variables per step, using the *entire* CP-SAT solver as the repair engine, which can find the best possible repair for that chunk, respecting every constraint that touches it, not just a locally best single move. A neighbourhood of size one, repaired by a real solver instead of a hand-written rule, would in fact reduce to something close to min-conflicts itself – LNS is a strict generalization, not an unrelated technique.

Common Pitfall

A common misconception is treating a larger neighbourhood as automatically better. It is not free: a neighbourhood of k variables hands CP-SAT a sub-problem whose own search cost grows with k, so an unnecessarily large neighbourhood can make each repair round itself slow, even though it may converge in fewer rounds. Choosing a neighbourhood size is a genuine trade-off between the cost of one round and the number of rounds needed, not a "bigger is strictly better" knob.

4.2.3 Choosing a neighbourhood

neighbourhood (in LNS): The subset of variables chosen to be relaxed and re-solved together in one destroy/repair iteration.

The **neighbourhood (in LNS)** is simply the chosen subset of variables a given LNS round relaxes and re-solves together – and which variables end up in it is a genuine design choice, not an afterthought.

A purely random subset of variables is the simplest destroy rule, and a reasonable default – it needs no problem-specific insight at all. A *targeted* destroy rule, relaxing specifically the variables currently involved in violations, plus a few extra for search room, often converges faster, because it spends the repair step's effort exactly where it is most needed instead of gambling on a random subset that may not even touch the current trouble spots.

Tip

When you are unsure which destroy rule to use, start with the targeted rule for the variables that already appear in violations. Adding a small number of extra, currently uninvolved variables (two or three, in this week's code) gives the repair step a little more room to maneuver than the bare minimum, without inflating the sub-problem's own solving cost.

4.2.4 Worked instance 1: large N-queens as repair

The first worked instance reuses the N-queens problem, this time reframed as a genuine repair task: start from a random complete assignment for $N = 50$ (one queen per row, a random column for each), count its attacking pairs informally as the quantity to drive toward zero, and repair it with LNS. (This "how many constraints does the current assignment violate" count is used here only operationally, as a concrete number for local search and LNS to improve step by step – the formal vocabulary for it, weighted CSP or Max-CSP, is Week 8's job, not this week's.)

The initial assignment and the destroy step are both short and direct:

```
rng = random.Random(seed)
columns = [rng.randrange(n) for _ in range(n)]
```

```
relaxed_rows = rng.sample(range(n), k)
```

The repair step is where CP-SAT enters: it builds a small model over just the $k = 8$ relaxed rows, with pairwise `AllDifferent` constraints among them (no two relaxed queens may share a column or a diagonal) plus constant-valued inequality constraints against every fixed row, then solves and keeps the proposal only if the total conflict count did not increase:

```
model.AddAllDifferent([relaxed_vars[r] for r in relaxed_rows])
model.AddAllDifferent([relaxed_vars[r] + r for r in relaxed_rows])
```

```
model.AddAllDifferent([relaxed_vars[r] - r for r in relaxed_rows])
for row in relaxed_rows:
   for fixed_row in fixed_rows:
      fixed_col = columns[fixed_row]
      model.Add(relaxed_vars[row] != fixed_col)
      model.Add(relaxed_vars[row] + row != fixed_col + fixed_row)
      model.Add(relaxed_vars[row] - row != fixed_col - fixed_row)
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
   return columns
proposal = list(columns)
for row in relaxed_rows:
   proposal[row] = solver.Value(relaxed_vars[row])
return proposal if count_conflicts(proposal) <= count_conflicts(columns) else
    columns
```

The outer loop repeats destroy and repair for a fixed number of rounds:

```
for round_number in range(1, rounds + 1):
   relaxed_rows = lns_destroy(n, k, rng)
   columns = lns_repair(columns, relaxed_rows, n)
   conflicts = count_conflicts(columns)
   print(f"Round {round_number}: {conflicts} conflicts")
```

The real captured run, 30 rounds with a fixed seed, drives the conflict count from an initial 64 down to zero well before the round budget runs out:

Expected Output

```
Round 0: 64 conflicts
Round 1: 50 conflicts
Round 2: 29 conflicts
...
Round 15: 1 conflicts
...
Round 28: 1 conflicts
Round 29: 0 conflicts
Round 30: 0 conflicts
```

Notice the plateau between round 15 and round 28: several consecutive rounds relax a random neighbourhood that never happens to touch the one remaining conflicted pair of rows, so the conflict count simply holds – a real, honest illustration of why the destroy rule's choice of which variables to relax matters as much as the repair step's own strength.

4.2.5 Worked instance 2: the Golomb ruler

Unlike large N-queens, this week's second worked instance is a genuine minimization problem, not a repaired satisfaction task. A **Golomb ruler** is a set of marks placed at distinct integer positions such that every pair of marks measures a different distance from every other pair.

Golomb ruler: A set of marks at distinct integer positions on a ruler such that every pair of marks measures a different distance from every other pair, commonly posed as minimizing the ruler's total length for a fixed number of marks.

The classic question about a Golomb ruler is the shortest one achieving this for a fixed number of marks, its **optimal Golomb ruler (OGR)**.

optimal Golomb ruler (OGR):

Analogy

A Golomb ruler resembles spacing a set of exam start times so that no two pairs of exams share the same time gap. Every pair of start times must be a different distance apart from every other pair, so that "the exams exactly 45 minutes apart" always refers to one unambiguous pair – exactly the ruler's "every pairwise distance distinct" rule, applied to a timetable instead of a physical ruler.

The base model fixes the first mark at 0, forces the marks into strictly increasing order, and collects every pairwise distance into one list constrained by a single `AllDifferent`:

```
marks = [model.NewIntVar(0, upper_bound, f"mark_{i}") for i in range(order)]
model.Add(marks[0] == 0)
for i in range(order - 1):
    model.Add(marks[i] < marks[i + 1])
```

```
distances = []
for i in range(len(marks)):
    for j in range(i + 1, len(marks)):
        distance = model.NewIntVar(1, upper_bound, f"dist_{i}_{j}")
        model.Add(distance == marks[j] - marks[i])
        distances.append(distance)
model.AddAllDifferent(distances)
```

Minimizing the last mark's position and solving to proven optimality for 8 marks reproduces a known combinatorial-design literature value (34), a standard ground-truth result playing the same role Week 1's OEIS A000170 played for 6-queens:

```
model.Minimize(marks[-1])
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
length = solver.Value(marks[-1])
positions = [solver.Value(mark) for mark in marks]
print(f"Status: {solver.StatusName(status)}")
print(f"Optimal length for {order} marks: {length}")
print(f"Marks: {positions}")
```

Expected Output

```
Status: OPTIMAL
Optimal length for 8 marks: 34
Marks: [0, 1, 4, 9, 15, 22, 32, 34]
```

LNS enters once a deliberately *non-optimal* starting ruler is on the table. A simple greedy construction – place each new mark at the smallest position that keeps every distance seen so far distinct – gives a valid but suboptimal ruler quickly:

```
marks = [0]
seen_distances: set[int] = set()
for _ in range(order - 1):
    candidate = marks[-1] + 1
    while True:
        new_distances = {candidate - mark for mark in marks}
        if len(new_distances) == len(marks) and not (new_distances &
            seen_distances):
```

```
            break
        candidate += 1
    marks.append(candidate)
    seen_distances |= new_distances
```

From that greedy start, each LNS round relaxes a small number of marks and re-solves just that sub-model, keeping the result only if the length did not increase:

```
for round_number in range(1, rounds + 1):
    relaxed = rng.sample(range(1, order), neighbourhood_size)
    proposal = _repair_relaxed_marks(marks, relaxed, order, upper_bound)
    if proposal is not None and _ruler_length(proposal) <= _ruler_length(marks
        ):
        marks = proposal
    print(f"Round {round_number}: length {_ruler_length(marks)}, marks {marks}
        ")
```

An exhaustive check over every 2-mark subset of this specific greedy start showed that none of them can improve the length in a single repair round – the six marks left fixed already pin the pairwise distances too tightly – so the real captured run relaxes 3 marks per round instead, the smallest neighbourhood size for which this file's own LNS run genuinely shrinks the ruler:

Expected Output

```
Round 0: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 1: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 2: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 3: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
...
Round 15: length 41, marks [0, 1, 11, 13, 18, 32, 38, 41]
```

The greedy start's length, 44, shrinks to 41 by round 3 and holds there for the remaining rounds – a genuine, non-trivial improvement over the greedy construction, though this particular 15-round, 3-marks-at-a-time run does not reach the base model's proven optimum of 34. That gap is not a bug: LNS never claims to reach an optimum on its own within a fixed round budget, only to improve on where it started, exactly the same honest limitation Unit 1's min-conflicts carries.

3 • Participation **Now You Try**

A large-N-queens LNS run has just produced a complete assignment for $N = 50$ with 6 remaining conflicts, all of them involving queens in rows 12, 13, 27 and 41 (each of these rows attacks at least one other row in this set; no other row is involved in any conflict). Describe which rows you would relax (destroy) for the next LNS iteration, which rows you would fix, and what the repair step's CP-SAT sub-model has to guarantee about the relaxed rows once it re-solves.

Feedback. Relax at least rows 12, 13, 27 and 41 – the rows actually in conflict; a reasonable destroy rule also adds a small number of extra, currently-uninvolved rows (two or three, picked at random) so the repair step has a little more room to maneuver than the bare minimum. Fix every other row's column value as a constant. The repair sub-model must place the relaxed rows so that none of them conflicts (shared column or shared diagonal) with each other *or* with any fixed row – expressed in CP-SAT as `AllDifferent`/pairwise constraints among the relaxed variables plus constant-valued inequality constraints against each

fixed row's column – with the goal of reaching zero conflicts among the relaxed set, or at least never doing worse than the 6 conflicts already present.

Check Yourself 4.2

A CP-SAT model has just found one complete large-N-queens assignment with some conflicts remaining. Describe, step by step, what one LNS iteration does next: which variables you relax, what you fix, and what the repair step's re-solve is required to guarantee about the relaxed variables.

5 • Follow-through — Transfer — Where Else?

The destroy/repair loop you just practiced by hand is the same idea behind OR-Tools' own routing solver (Week 14), which relies on large neighbourhood search as one of its core metaheuristics, and behind re-planning a schedule after a small change without re-solving it from scratch (Week 12's rostering) – only the domain changes; the loop itself is this week's, reused wholesale.

4.3 Restarts and Nogood Learning; Lazy Clause Generation

1 • Pre-instruction — Why It Matters

Every restart you might add to Week 3's backtracking search, on its own, throws away everything the previous attempt discovered – it can walk into the exact same dead end twice, or a hundred times. CP-SAT never does this by accident: every failure it hits is turned into a permanent rule ("this exact combination never works") before the next attempt begins, and this one habit is a large part of why CP-SAT is fast on problems your own Week 3 code can struggle with.

1 • Pre-instruction — Prior Knowledge

Week 3's backtracking and forward checking/MAC both revolve around a dead end – a domain wipeout discovered during search. This unit asks what a genuinely smart search does *immediately after* hitting that same kind of dead end, instead of simply undoing the last decision and moving on chronologically, which is exactly what plain chronological backtracking, as built in Week 3, does.

Units 1 and 2 both left systematic search behind entirely. This unit returns to it – Week 3's backtracking tree is still there, still building partial assignments one variable at a time – and asks what changes when that tree is allowed to abandon a bad branch outright and try again, this time remembering what went wrong.

4.3.1 Restarts, on their own

restart (search): Abandoning the current search branch entirely and beginning the search again from the root, typically with a different variable/value order or random seed.

A **restart (search)** abandons the current search branch entirely and begins again from the root, typically with a different value order or a different random seed, in the hope of avoiding a bad early branching decision the current branch is stuck committed to.

Definition: Restart (search)

Abandoning the current search branch entirely and beginning the search again from the root, typically with a different variable/value order or random seed.

On its own, with no memory kept between attempts, a restart is a gamble that can go badly wrong: it can walk straight back into the exact same dead end it just escaped, over and over, paying the full cost of rediscovering the same failure every single time.

Common Pitfall

A tempting but incorrect assumption is that restarting with a different random seed is, by itself, enough to make a stuck search productive again. It is not, unless the new random choices happen to avoid the earlier trouble spot entirely – and on a genuinely hard instance, they frequently do not. A restart with no memory is a bet on getting lucky; what actually fixes the underlying problem is remembering the failure at all, which is exactly what nogood learning, below, adds.

4.3.2 Nogood

Consider a toy 4-variable CSP, the spine example for the rest of this unit: branching $V_1 = a$, then $V_2 = b$, and only then discovering that V_3 has no legal value left, no matter what V_4 is later set to. The failure came entirely from the pair $V_1 = a \wedge V_2 = b$ together – V_3 and V_4 never mattered to it at all. That is exactly what a **nogood** records: a combination of variable assignments proven, by this discovered failure, to never be extendable to a solution. This particular failure justifies recording $\neg(V_1 = a \wedge V_2 = b)$.

nogood: A recorded combination of variable assignments proven, by a discovered failure, to never be extendable to a solution; recording it lets the search reject that exact combination immediately in the future.

Definition: Nogood

A recorded combination of variable assignments proven, by a discovered failure, to never be extendable to a solution; recording it lets the search reject that exact combination immediately in the future.

Analogy

A nogood resembles a note pinned to a shared lab-booking calendar: "Room 204 + Tuesday 9am + the microscope reservation – never works, already tried and failed." Pinned up in a place everyone on the team can see it, so no one wastes time proposing that exact combination again, no matter which of them is doing the booking.

4.3.3 Nogood learning

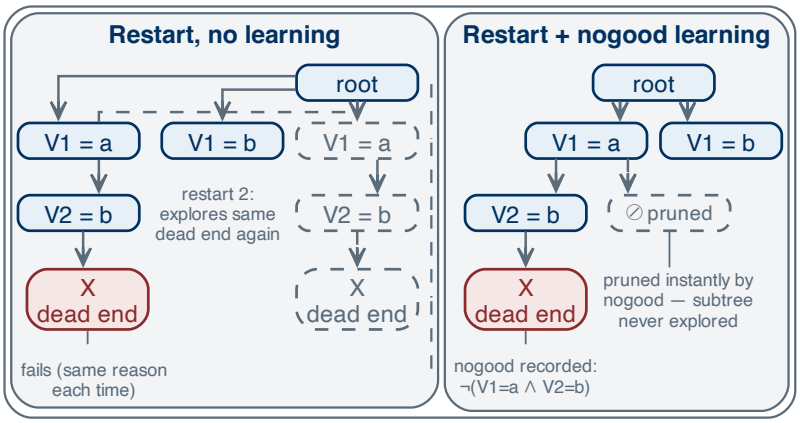
Nogood learning (clause learning) is the technique of recording a nogood every time search hits a failure, and using the growing set of recorded nogoods to prune later parts of the *same* search – including after a restart. Once $\neg(V_1 = a \wedge V_2 = b)$ is recorded, any later branch – in this attempt, or the next restart – that tries $V_1 = a$ followed by $V_2 = b$ fails *immediately*, without re-exploring V_3 or V_4 at all. The entire subtree the first failure took to discover is never paid for twice.

Nogood learning (clause learning): The technique of recording a nogood every time search discovers a failure, and using the growing set of recorded nogoods to prune later parts of the same search, including after a restart.

Analogy

Restarts combined with nogood learning resemble planning a group trip itinerary. A first attempt fails because two connections do not line up; a second attempt that remembers this does not repeat the exact same doomed combination – everything else about the second attempt can still be freely chosen, only that one failed combination is ruled out.

Figure 4.3 contrasts two small search trees on this exact spine example, side by side: one restarting with no memory at all, repeating a full failing subtree on its second attempt, against one restarting with the recorded nogood, where the second attempt is pruned instantly.



Same failure, discovered once — the right panel never pays for it twice.
Illustrative — schematic search trees (toy example), not a captured solver log.

Figure 4.3: Restart without learning repeats a full failing subtree on its second attempt; restart with a recorded nogood prunes the identical failure instantly, on the same $V_1 = a, V_2 = b$ example this unit worked through by hand.

4.3.4 Lazy clause generation

Lazy clause generation (LCG): A hybrid search technique in which each constraint propagator can explain, on demand, why it removed a value or caused a failure, as a clause; those explanations feed a SAT-style clause-learning engine.

Lazy clause generation (LCG) is the machinery that lets a solver like CP-SAT do this kind of learning automatically, from *every* kind of constraint it supports, not only from a hand-encoded Boolean formula. Each constraint's own propagator can explain, on demand, exactly why it removed a value or caused a failure, as a clause over the decisions that caused it; those explanations feed a SAT-style clause-learning engine, exactly the engine behind the nogoods this unit already worked through by hand. This combines CP's rich global constraints (Week 2) with SAT's efficient nogood-recording machinery, and is a large part of why CP-SAT is fast in practice. What matters here is the idea and what it buys – learning from any propagator's failure, not only from a hand-encoded SAT formula – not the clause-learning algorithm's full derivation, which is out of scope here.

Complexity (big-O)

Nogood learning does not change a CSP's worst-case complexity – in the worst case, a search can still record exponentially many nogoods before find-

ing a solution or proving infeasibility. What it changes is *typical-case* behaviour: on real instances, a modest number of learned nogoods often prunes an enormous number of branches that would otherwise be re-explored, which is exactly why restarts, on their own, can hurt, while restarts combined with nogood learning almost never do.

The real captured search log for a Golomb-ruler order-9 optimality proof (the same instance Unit 4 reuses for its own single- versus multi-worker comparison) shows this activity concretely. CP-SAT's per-worker search statistics report a real restart count and conflict count for this run:

Expected Output

```
Search stats    Bools  Conflicts  Branches  Restarts  BoolPropag
↪  IntegerPropag
       'main':    120      8'573    12'008     1'139     119'754
       ↪  627'411
```

and the solver's own end-of-run summary confirms the same numbers:

Expected Output

```
conflicts: 8573
branches: 12008
...
restarts: 1139
```

Over 1,139 restarts, the single "main" worker recorded 8,573 conflicts (failures) while proving the order-9 ruler's optimal length of 44 – each of those conflicts is exactly the kind of failure this unit's $V_1 = a, V_2 = b$ toy example worked through by hand, turned into a nogood before the next restart begins, one restart's worth of learning feeding directly into the next.

3 • Participation **Now You Try**

During backtracking on a 4-variable CSP you branch $V_1 = a$, then $V_2 = b$, and only then discover V_3 has no legal value left no matter what V_4 is set to – the failure came entirely from the pair $V_1 = a, V_2 = b$ together. Write the nogood this failure justifies recording, and explain, for the very next restart of the same search, which branch it would prevent from being retried.

Feedback. The nogood is $\neg(V_1 = a \wedge V_2 = b)$ – "$V_1 = a$ and $V_2 = b$ together are never both allowed," independent of V_3's or V_4's values. On the next restart – or any later branch within the same search – as soon as the search tries $V_1 = a$ followed by $V_2 = b$, the recorded nogood fails that branch immediately, without re-exploring V_3 or V_4 at all, saving the entire wasted subtree the original failure took to discover.

Check Yourself 4.3

Two students backtrack through the same hard CSP. Student A restarts from scratch with a different random value order every time the search fails, remembering nothing between attempts. Student B does the same restarts, but addi-

tionally records, as a nogood, each partial assignment that caused a dead end. Explain, using the idea of a nogood, why Student B's approach converges faster on average.

5 • Follow-through **Transfer — Where Else?**

Nogood learning is exactly why every "CP-SAT versus a hand-written Week 3 backtracker" comparison you run for the rest of this course tends to favour CP-SAT so heavily – it is not raw speed, it is learning. The same underlying habit – record a specific, reusable reason for a failure instead of only undoing the decision that caused it – is a general debugging and engineering practice as well, and Week 15's infeasibility/explanation work (unsat cores, IIS) formalizes this exact idea for a different purpose: explaining a failure to a human, not just pruning a search.

4.4 Portfolios and Parallel Search

1 • Pre-instruction **Why It Matters**

So far this week you used one search strategy at a time – local search, then LNS, then restarts with learning. CP-SAT does not force that choice: set one parameter, and it runs several of these strategies at once, on separate CPU cores, and simply keeps whichever result comes back first (or best) – several attempts for the price of waiting for the fastest one.

1 • Pre-instruction **Prior Knowledge**

Week 3's own search-parameters unit set `num_search_workers` as an opaque performance knob without explaining what it launched – Units 1–3 of this week (local search, LNS, restarts and nogood learning) are exactly the ingredients this unit reveals as running underneath it, all at once.

That opaque knob from Week 3 is finally opened. Everything this unit needs is already on the table: local search's willingness to trade completeness for speed (Unit 1), LNS's use of a full solver as a repair engine (Unit 2), and restarts with nogood learning (Unit 3) – this unit does not add a new search technique so much as it explains how several of the ones you already know run side by side.

4.4.1 Portfolio search

Portfolio search: Running several different search strategies concurrently on the same problem instance and keeping the result of whichever strategy finishes first (or proves the tightest bound), instead of committing to one strategy in advance.

Portfolio search runs several different search strategies concurrently on the *same* problem instance, and keeps the result of whichever strategy finishes first – or, for an optimization problem, proves the tightest bound – instead of committing to one strategy in advance and hoping it happens to fit this particular instance well.

Definition: Portfolio search

Running several different search strategies concurrently on the same problem instance and keeping the result of whichever strategy finishes first (or proves

the tightest bound), instead of committing to one strategy in advance.

Analogy

Portfolio search resembles asking several travel agents to plan the same trip at once, each using a different strategy – cheapest fare first, fewest layovers first, fastest arrival first. Whichever comes back with a usable itinerary first, you take it – you never needed to know in advance which strategy would win, and the other agents' unfinished work is simply set aside.

4.4.2 What CP-SAT's workers actually are

search worker: One concurrently running search process in a portfolio search; each worker may use a different strategy, and workers typically share learned information (such as nogoods) with each other while running.

Setting `solver.parameters.num_search_workers = k` does *not* simply run the same search k times faster on k copies of identical work. It launches, roughly, k different concurrent **search workers** – mixing systematic-search variants with different variable/value orders and restart policies (Unit 3), and dedicated large neighbourhood search workers (Unit 2). Workers share what they learn – in particular, learned nogoods (Unit 3) and objective/dual bounds – with each other while running, so a nogood one worker discovers can prune another worker's search too. Setting this one parameter retroactively explains Week 3's own `num_search_workers` parameter, which you already used there as an opaque performance knob.

Figure 4.4 draws this architecture: one `Solve()` call fans out into several differently-strategized workers, all reading from and writing to a shared learned-nogood/bound pool, all feeding into the single result you actually receive.

4.4.3 Practical guidance, and an honest limit on the speed-up

A sensible starting point is one worker per available CPU core, but returns diminish well before every core is saturated, for two concrete reasons. First, not every worker's job is to speed up the specific answer you are waiting for – an LNS worker is chasing a better solution, not necessarily proving optimality faster, so on an optimization proof some of the k workers may contribute nothing directly to the bound that ends the search. Second, coordinating shared information (clause and bound sharing) has its own cost, and different workers suit different instances, so adding workers is not a clean multiplicative speed-up.

Common Pitfall

It is tempting to assume that `num_search_workers=8` means "roughly 8× faster." The two reasons above are exactly why that assumption fails in practice, and this week's own captured numbers below confirm it directly, on a real proof rather than an assumed one.

The real captured wall-clock comparison, single- versus 8-worker, for the same order-9 Golomb-ruler optimality proof Unit 3 already examined via its search log, reports:

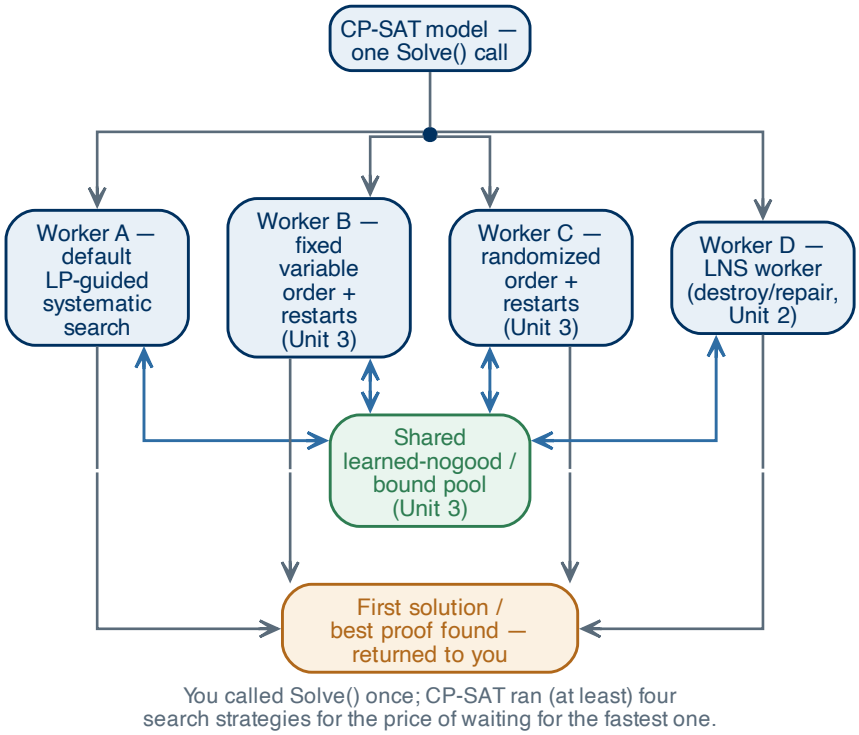


Figure 4.4: What `num_search_workers` actually launches: several concurrent workers with different strategies (systematic-search variants, an LNS worker), sharing a learned-nogood/bound pool, feeding one returned result.

Expected Output

```
num_search_workers=1        wall_time=0.1707s  length=44  status=OPTIMAL
num_search_workers=8        wall_time=0.0443s  length=44  status=OPTIMAL
speed-up (1-worker time / 8-worker time): 3.86x
```

Both runs prove the same optimal length, 44 – parallel search changes only the time to find that proof, never the answer itself – and the real speed-up, 3.86×, is genuinely helpful but clearly short of the 8× a naive "k workers, k× faster" guess would predict, exactly the outcome the two reasons above lead you to expect. Figure 4.5 shows the same two numbers as a bar chart.

4.4.4 Reproducibility caveat

Every other script this week set `solver.parameters.random_seed = ...` to make a *single*- worker run's search order deterministic, used throughout this week's own code for reproducible step-by-step traces. A multi-worker run is not guaranteed to reproduce the identical solution or timing across runs, even with the same seed, because the workers' relative progress depends on real wall-clock scheduling – which worker happens to finish its current sub-task first can vary from run to run on the same machine. State this plainly rather than promising a determinism the parallel case cannot deliver: a fixed seed controls each worker's own internal choices, not the race between them.

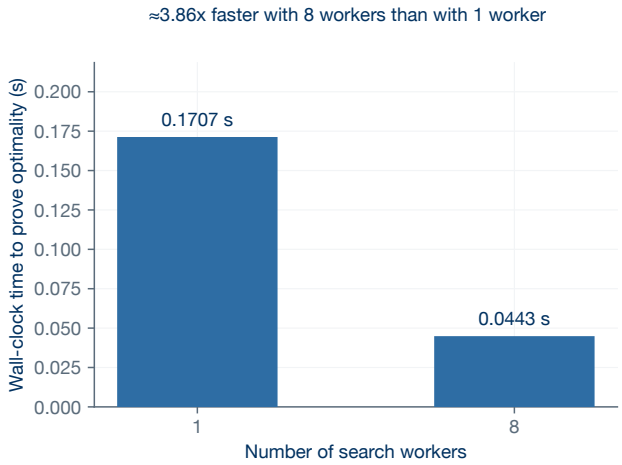


Figure 4.5: Real captured wall-clock time for the order-9 Golomb-ruler optimality proof, `num_search_workers=1` versus `num_search_workers=8` – a genuine speed-up, clearly short of $8\times$.

3 • Participation **Now You Try**

You solve the same Golomb-ruler optimality proof three times, using your own real captured wall-clock times: once with `num_search_workers=1`, once with `num_search_workers=4`, once with `num_search_workers=8`. Is the 8-worker run's speed-up over the 1-worker run closer to $8\times$, or clearly less than $8\times$? Give two concrete reasons, from this unit, why parallel CP-SAT search is not a simple linear speed-up.

Feedback. Read your own `ciktilar/portfolio-timing.txt` – there is no single fixed number to copy, but the speed-up is typically clearly less than $8\times$ (occasionally close to linear on an instance well suited to parallelism, but that is not the general case; this week's own captured run above reached $3.86\times$). Two reasons from this unit: (1) not every worker's task directly shortens the specific answer you are waiting for – some workers, such as LNS workers, are chasing solution quality rather than helping prove the bound that ends the search; (2) workers share information (learned nogoods, bounds) and different workers suit different instances, so added workers give diminishing, not multiplicative, returns.

Check Yourself 4.4

You solve the same large Golomb-ruler proof twice, once with `num_search_workers=1` and once with `num_search_workers=8`, and get two runs that both prove the same optimal length but take different real amounts of time. Explain why the multi-worker run is not simply "$8\times$ faster" in practice, giving two concrete reasons from this unit.

5 • Follow-through **Transfer — Where Else?**

Portfolio thinking – try several strategies at once, keep the best/first result – generalizes past CP-SAT's own parameter: Week 12's CP–MIP hybrids and Week 15's solver/model benchmarking both compare or combine multiple solving strategies on purpose, and the general engineering habit of running several approaches in parallel rather than betting everything on one "best" method up front shows up whenever the best method is not known ahead of time – which, in practice, is most of the time.

Chapter Summary

Summary

This chapter traded Week 3's completeness guarantee for speed on instances too large for a search tree to handle exhaustively, one mechanism at a time. Unit 1 defined local search as a search method that starts from a complete, possibly invalid assignment and repeatedly repairs it – min-conflicts picks a conflicted variable and reassigns it to the value with fewest conflicts – and showed how it can get stuck at a local minimum, with tabu search and simulated annealing as two ways to escape one; the price for all of this speed is that local search can never prove a CSP infeasible. Unit 2 generalized that same start-from-a-solution, repair-it habit from one variable at a time to a whole relaxed neighbourhood at once, handing each repair step to CP-SAT itself: worked on large N-queens (conflicts driven from 64 to 0 over 30 rounds) and on the Golomb ruler (a greedy, non-optimal 44-length start shrunk to 41 by LNS, short of the base model's proven optimum of 34 – LNS improves, it does not by itself guarantee reaching an optimum within a fixed round budget). Unit 3 returned to systematic search and showed that a restart without memory can walk back into the same dead end repeatedly, while a restart that records each failure as a nogood – $\neg(V_1 = a \wedge V_2 = b)$, in this unit's worked toy example – never pays for the same failure twice, with lazy clause generation as the machinery that extracts this kind of explanation from every constraint CP-SAT supports, not only from hand-written clauses; a real captured search log recorded 1,139 restarts and 8,573 conflicts while proving an order-9 Golomb ruler's optimal length of 44. Unit 4 closed the chapter by opening `num_search_workers` itself: several differently-strategized concurrent workers – systematic-search variants with different orders and restart policies, and dedicated LNS workers – sharing learned nogoods and bounds while running, with a real captured 1-worker-versus-8-worker comparison on the same order-9 proof showing a genuine but sub-linear 3.86× speed-up, for two concrete reasons: not every worker's work shortens the specific proof you are waiting for, and coordinating shared information has its own cost. Next week keeps every technique this chapter built, unchanged, but asks a different question that never depends on how you search at all: does a CSP's own constraint-graph shape make it tractable, independent of the search algorithm pointed at it? This chapter's restarts, nogood learning and local search improve typical-case performance without touching that worst-case picture – Week 5's bounded-treewidth and symmetry results are what explain the worst case, and the two chapters' answers are complemen-

tary, not competing.

Check Yourself Answers

Question (Unit 1 – min-conflicts, local minima and simulated annealing). Min-conflicts only ever looks one variable-reassignment ahead and always takes the best available local move, so – unlike systematic search – it never builds or explores a search tree and can never prove infeasibility; it simply repairs a complete assignment until zero violations remain or its step budget runs out. It can plateau at a local minimum: a complete assignment where every single-variable move is equal to or worse than the current state, even though a genuine solution may still exist elsewhere in the space of assignments – min-conflicts has no way out on its own once it reaches one. Simulated annealing fixes this by sometimes accepting a strictly worse move, with a probability that starts high and cools over time, letting the search walk out of that local minimum at the cost of occasionally leaving a good state temporarily worse before it improves again.

Question (Unit 2 – one LNS iteration on large N-queens). Relax a chosen neighbourhood – the variables already involved in current conflicts, plus a few extra for search room – and fix every other variable at its current value. The repair step's CP-SAT sub-model must place the relaxed variables so that none of them conflicts (shared column or shared diagonal) with each other or with any fixed variable, accepting the result only if it does not increase the total conflict count already present; if the repair makes things worse, the previous assignment is kept instead. This is exactly the loop this week's own `large_nqueens_lns.py` runs 30 times, real captured rounds carrying the conflict count from 64 down to 0.

Question (Unit 3 – Student A versus Student B's restarts). Every one of Student A's restarts can walk straight back into a dead end already discovered on an earlier attempt, since nothing is remembered between attempts – each rediscovery pays the full cost of re-exploring the subtree that reveals the same failure again. Student B's recorded nogoods let any later attempt reject that exact combination of assignments immediately, without re-exploring the subtree that originally revealed it. Over many restarts, the same total set of failures gets discovered once each under Student B's approach, rather than repeatedly under Student A's – exactly the gap this week's own captured search log illustrates at scale, 1,139 restarts feeding a shared, growing pool of 8,573 learned conflicts rather than 1,139 independent, memoryless attempts.

Question (Unit 4 – why 8 workers is not $8\times$ faster). Two concrete reasons. First, not every one of the 8 workers is doing the same kind of useful work – some run systematic-search variants aimed directly at proving the bound that ends the search, while others, such as LNS workers, are chasing a better solution instead, so only part of the parallel effort directly shortens the specific proof you are waiting for. Second, workers share learned information (nogoods, bounds) and different workers are better suited to different instances, so coordination overhead and uneven usefulness across workers mean added workers give diminishing, not linear, returns. This week's own real captured numbers bear this out directly: a single worker proved

the order-9 Golomb ruler's optimal length of 44 in 0.1707 s, and 8 workers proved the same length in 0.0443 s – a genuine 3.86× speed-up, clearly short of 8×.